

---

## The Publishing Accelerator Toolkit (PacTk)

---

Amy Neustein, Ph.D.  
CEO and Founder  
Linguistic Technology Systems  
Fort Lee, NJ  
917-817-2184  
amy.neustein@verizon.net

*A Collection of Software, Compiler, and Runtime Tools  
for Managing and Visualizing Multimedia Data Sets*

Overview

In contemporary publishing, the standard workflow includes **XML**-based formats such as **JATS** or **BITS** from which are compiled reader-visible documents such as **PDFs**. These intermediate files are distinct from those that authors submit directly, and they can be computationally accessed via generic **XML**-oriented protocols such as **SAX** (Simple **API** for **XML**), **XQUERY**, and **XQSE** (**XQUERY** scripting extension). Unfortunately, **XML** itself is not explicitly built for representing textual data, and examining **JATS** or **BITS** files via generic **XML** technology renders certain query and editing tasks difficult or impossible to implement. The **PDF** format is similarly opaque because internal **PDF** representations of document text include many anomalies due to ligatures, hyphenation, font specs, and other layout-related discrepancies between **XML** inputs and user-visible output.

Given the limitations of **XML** for natural-language documents, numerous competing formats have been developed that provide alternatives or extensions to **XML**. These include **TEI** (Text Encoding Initiative), **TAGML** (Text-as-Graph Markup Language), **BIOC** (a biomedical text mining standard defined by PubMed Central), and Research Object Bundles (a Semantic-Web based specification that converts both text and data sets). With a proliferation of different text-encoding standards, query and traversal protocols that are developed for one format (e.g., **JATS** and its derivatives) cannot be directly applied to others. For example, the **XML** parent-child element relationship is replaced in **TAGML** with two different containment styles (logical and extensional), such that **DOM** and event-based parsers targeted at the simpler **XML** hierarchy must be reimplemented for the **TAGML** system. From the perspective of **PacTk**, text-encoding evinces a spectrum of distinct *topomorphic regimes* where a proliferation of incompatible document structures prohibits the emergence of common query, traversal, and metadata formats.

One important variation amongst text-encoding formats is the level of granularity for most textual elements. In **JATS**, **BITS**, and most **JSON** encodings, the fundamental discursive unit is the individual paragraph. Elements on smaller scales (such as sentences or block quotes) are notated haphazardly at best. **L<sup>A</sup>T<sub>E</sub>X** employs reasonable sentence-boundary heuristics (that can be overridden via macros when necessary), but sentence objects are not expressed in any systematic manner (e.g., via auxiliary files) except for intermediate spacing. In **TEI** and **BIOC**, individual sentences *can* be represented, but this is not mandatory (thus **s** and **sentence** tags are available but not guaranteed to be present in any particular document). By contrast, **PacTk** is explicitly built around sentence-level units, specifically a “topomorphic sentence-level object model” (**TSOM**). When all regular text in a natural-language document is explicitly contextualized in a delineated sentence object, many editing, indexing, and user-interface tasks become feasible (as outlined in the next section).

In contrast to the other formats mentioned here, **PacTk** is not a single file format but rather a collection of code libraries and classes, with data types representing specific textual elements (sentences, paragraphs, **PDF** pages, annotations and comment-boxes, etc.). Query and traversal protocols modeled via **XQUERY**, **DOM**, and so on, become expressed by object methods and operators exposed to **C++** code, for

example (**PacTk** is thus “topomorphic” in the sense that navigation between objects is determined by structural properties of divergent document formats, that manifest in competing notions of individuation for individual structure-sites and their interrelationships). In short, the **PacTk** object system synthesizes distinct structuring protocols found in competing formats such as **JATS**, **TAGML**, and **BIOC**, allowing publishers’ workflows to encompass each of these formats (and others) without sacrificing compatibility. **PacTk** is particularly aimed at publishers who feel constrained by the limitations of **JATS** or **BITS** but are reluctant to try alternatives (**TAGML**, **BIOC**, etc.) due to minimal software-ecosystem support and the lack of standardized **XQUERY** and **DOM**-style interfaces.

The remainder of this document will discuss how **PacTk** can streamline editing, indexing, and document-preparation tasks, as well as improve User Experience for human readers. Later sections will also address how **PacTk** can contribute to broader projects where text documents are published together with data sets and/or multimedia presentations. Overall, **PacTk** presents a unified model and coding interface for text queries operating in diverse contexts, including **PDF** viewers, search engines, and document-preparation code. Similarly, software employing **PacTk** to navigate through document objects can leverage interop metadata across different formats, such as **XML** elements and **PDF** page coordinates, that are typically unavailable in other publishing tools.

---

### *Features for Editing, Indexing, and PDF Viewers*

---

The computational foundation of **PacTk** is the collection of classes that represented individual sentences and other textual elements in the **TSOM** family (**PDF** pages, annotations, footnotes, block quotes, edits or changes across document versions, etc.). In order to initialize these objects, **PacTk** provides a distinct input format called **GTagML**, or “grounded” **TAGML**. The term “grounded” reflects how certain structure-sites may be specifically marked as serializing a typed object: i.e., tags, attributes, or text nodes in the neighborhood of a site supply the data fields needed to initialize an object-instance during deserialization. In other respects **GTagML** follows most of the principles advocated by **TAGML**, with novel extensions; for example, **GTagML** encourages the exploitation of Unicode Private Use Planes to disambiguate visibly similar but logically distinct characters/glyphs (e.g., periods may embody punctuation, acronyms, or abbreviations).

In source files, **GTagML** is loosely based on Markdown, so authors familiar with Markdown from more informal contexts (e.g., social media, chat rooms, **GIT** documentation, etc.) will recognize many of the **GTagML** styling commands. **GTagML** sources might be provided by authors directly, or derived from author inputs by digital copy editors or others on the publishing workflow. To support **GTagML**, **PacTk** provides a new **C++** parsing library that is readily extensible (more so than parsers for **XML**, **JSON**, and so forth), allowing publishers to customize the **GTagML** format with new tags, syntax, and data structures specific to their in-house styles and disciplinary focus areas (e.g., specialized tags/data for different scientific fields — chemistry, physics, medicine, etc. — or whatever subject-areas are addressed by individual publishers, journals, and book series). Alternatively, **TSOM** objects can be compiled from sources such as **JATS** or **BITS** directly.

For workflows that feature **JATS**, **BITS**, or **L<sup>A</sup>T<sub>E</sub>X**, **TSOM** object bundles would be used as adjunct data sources through which edits, queries, and other computational artifacts would be routed. **PacTk** data can then be mapped both to archiving formats (like **XML**) and camera-ready generators (like **L<sup>A</sup>T<sub>E</sub>X**), but with a common **TSOM** background these two different representational facets may be integrated (for example, **XML** elements mapped to corresponding **PDF** page coordinates). In effect, **PacTk** serves to triangulate human-visible presentations (like **PDF**) with behind-the-scenes **JATS**-style formats. Analyses that are currently implemented against **XML** structures may be more efficiently coded via **PacTk** objects, with concrete benefits in areas such as copy-editing and indexing.

### PacTk for Editing

**PacTk** has an advanced query framework to help find specific locations in text where edits may be warranted (e.g., to comport with journal or book-series conventions) and to correctly review and process such edits. Here are some features:

**Contextual Filters for Text Locations** Often it is useful to restrict the scope of text searches to characters with specific presentation features (italics, alternative typefaces, citations), discursive roles (inline/block quotes, footnote text, section/subsection titles), or semantic significance (acronyms, annotations, proper/place names).

Implementing such filters via **XPATH**/tag restrictions can be inaccurate, because relevant filter criteria may be orthogonal to **DOM** hierarchies. For example, **XML** or **L<sup>A</sup>T<sub>E</sub>X** code may wish to wrap elements such as footnotes or block quotes in custom environments, to ensure desired pre- or post-processing. Content that is functionally footnote or quoted text, therefore, may not necessarily be indicated with those specific tag names. The **PacTk** system allows programmers to define lists of boolean parameters that apply to text-sequences and then activate those flags automatically in specific tag contexts.

In general, **PacTk** supports queries where ordinary textual restrictions may be augmented with filters for a wide variety of layout and discursive contexts. Such queries could then be used for editing tasks that should be localized to specific contexts, or where a user wants to find a specific piece of text known to have a given context.

**Sentence-Level Queries** Siting queries relative to sentence boundaries can improve precision and express query rationales more succinctly. For instance, the proper punctuation around a word or phrase often depends on whether that content appears at the beginning,

middle, or end of a sentence. Some query expressions may similarly depend on stipulating that two or more search targets (e.g., two proper names) occur in the same sentence.

Because **PacTk** recognizes almost all text as contained inside sentence objects, queries dependent on sentence boundaries can be directly evaluated on **TSOM** data (such boundaries are unambiguously notated via input sources, rather than detected approximatively using Natural Language Processing). **PacTk** likewise models “topomorphic” navigational relations between sentences, such as between adjacent siblings of parent objects (typically paragraphs) and also nestings where a multi-sentence footnote, for example, occurs in a parent-sentence context, or where a block quotes functions in part as a logical child of the sentence that introduces it.

**PDF Page Coordinates** Human readers post-publication — plus authors and copy-editors beforehand — typically view documents in **PDF** format for normal reading, even if edits/changes are made on intermediary files. As a result, a bitmapped page-image is the primary medium through which users view single publication pages. Failure to interoperate these images with structured text formats is a significant hindrance to efficient document preparation and editing.

By design, **PacTk** supports post-processing algorithms within pipelines whose formats can generate **PDF**-coordinate metadata, such as **LaTeX** **pgfsavepos** (and auxiliary **write**) commands. Page coordinates for significant discursive locations (e.g., sentence and paragraph boundaries, footnote markers, keyphrases and named entities, citations) can thereby be exposed to **PDF** viewers which could accordingly implement novel front-end features such as overlays, context menus, and dialog windows positioned nearby relevant content. For example, a context menu action may include copying the full text of a current sentence to clipboard, based on the screen position where the menu is activated; semitransparent overlays could similarly show the extent of individual sentences during mouseover events.

**PDF Comments and Annotations** In addition to sentence objects, **PacTk** classes model data specifically relevant to the **PDF** front-end milieu, including single pages, links/anchors, and comments/annotations. Objects such as comment-boxes may thereby be leveraged systematically in an editing/review pipeline.

In particular, **PacTk** supports construction of hashtag- or handle-style controlled vocabularies that can annotate **PDF** comments introduced by copy editors or typesetters. Many modifications considered during the editing process follow common patterns, such as proper use of commas around subordinate clauses, or correct formatting of ellipses and bracketed content within inline/block quotes. Assuming that editors consistently include notations for recurring patterns, **PacTk** code can filter queries specifically to **PDF** comment boxes which indicate a specific editing category. Index codes within **PDF** comments can likewise indicate which individual sentence surrounds the annotation, so that the **PDF** view may be cross-referenced against input sources, which facilitates review and processing of manuscript changes.

For similar reasons, **PacTk** supports abstractions such as **XML** custom character entities and **LaTeX** macros. Many familiar textual components — ellipses, m- and n-dashes, quotation and footnote marks, etc. — are typically represented not via their visible characters but rather through standardized codes or glyphs. The problem is that systematic mappings are lacking when mixing different filetypes (e.g., **LaTeX** and **XML**, such as **JATS** or **BITS**) in a single workflow. **PacTk** has a flexible abstraction system that allows character and macro codes to propagate across different generated files.

A related problem is that abstraction codes should generally be opaque to human users in contexts such as comment boxes and copy-paste between software applications. **PacTk** therefore supports declarations for how custom entities/macros and Unicode Private Use Area codepoints should be rendered in plain-text contexts as well as structured formats such as **XML** and **LaTeX**.

Often users may prefer to initiate or review edits though **PDF** files directly, but in other cases plain-text files may be appropriate instead, without **GUI** overhead. With **PacTk**, for example, authors could receive two text files (or collections of files indexed, for instance, by chapter) summarizing changes between two manuscript versions. Via sentence-level filtering, such files could be organized such that only sentences that differ from one version to the next are included, with each sentence listed separately. This may be a more convenient setup for authors to review changes than working through typical **GUIs**. Of course, such breakdown is only possible when the text encoding systematically identifies individual sentences, as with the **PacTk**’s **TSOM** object model.

---

### ***PacTk for Indexing and Search Engines***

---

Authors might use several methods to compile an index. They can start with a list of important headings (names, concepts, etc.) and search for relevant passages in the manuscript. Alternatively, authors could read through a document and identify when a particular paragraph mentions a named entity suitable for indexing. Depending on representations or tag sets used (e.g., the **JATS** **string-name** element) annotated spans and citations may also form the provisional heading-list for an index.

Given these various index-preparation methods, it is useful to extend front-end tools with capabilities specific to indexing workflows, such as query matching and manipulating index entry objects. Structured formats like **JATS** do not provide sufficient computational tractability in themselves. For example, the most popular desktop **GUI** framework are the **Qt C++** libraries, which are the basis of cross-platform **PDF** viewers such as Poppler and **XPDF**. **PacTk** can seamlessly interoperate with **Qt**, exposing sentence texts as **QString** objects and

expressing **GTagML** parsing grammars as **QRegularExpression** instances, paired with lambda callbacks activated by the first matching rule (relative to the current parse-position). More precisely, **GTagML** has context-sensitive grammars that can include or skip over rule-tests via activated contexts; this allows sections of a **GTagML** file to utilize syntax different from the default. With respect to indexing, **GTagML** has its own notation for index entries (headings, locators, and “see also” style redirections), to serialize indexes in a human-readable format. Correspondingly, **PacTk** has classes specifically for index entries and associated data. These objects could be initialized from **GTagML** files or specialized windows floating atop **PDF** viewers.

Building indexes on the basis of native sentence and index-entry objects has certain benefits as compared to generic (e.g., **XML**) manuscript formats. These include:

**Separating Data from Presentation** Different publishers or book series may choose different visual styles for their indexes. Representing indexes behind the scenes via structured data objects allows indexes to be curated in abstraction from the specific layout chosen for final publications.

**PDF Context Menus** Authors/editors may prefer add new index entries or locators directly from **PDF** pages. **PacTk**-compliant viewers will read metadata that specifies which paragraphs are in scope at different page-coordinate regions, so paragraph ids for any location can be ascertained from mouse events (e.g., the dropdown point of a context menu).

**Multiple Match Strategies** Index locators are typically identified by matching text against entry-headings, but the parameters of these searches will vary by context. Sometimes locators are only relevant when the local sentence includes just the exact heading label, but in other places a desired match could include words in different order (e.g., either first-name/last-name or vice-versa), or selected words (e.g., last name only), or associated concepts with different language entirely (e.g., “The White House” in lieu of a president’s name). In the presence of a specific index entry, front-end displays can give users options to fine-tune which match strategies are appropriate for query text seeking locators specific to each entry.

**Index-Building GUI Components** Index entries are multi-faceted objects that naturally lend themselves to dedicated front-end displays, such as dialog windows. For example, entries typically include a heading label and then one or more locators (e.g., paragraph ids), then possible subheadings with their own locators, plus “redirectors” (*see* or *see also*) applied either to overall entries or to subheadings. This represents a lot of information to keep track of for each entry, and indexes can include hundreds of entries. Although indexes are eventually shown to readers as ordinary **PDF** pages (or similar formats), during document prep it is useful to subdivide indexes into individual **GUI** windows for each entry, with options to navigate between adjacent entries, as well as traverse on larger scales (e.g., one chapter to another) and to create new entries on the fly.

**Index-Guided Text Searching** Indexes are curated lists of headings and locators selected by authors/editors for relevance. Whenever a search is posed against a manuscript in any context — e.g., in the query bar of a **PDF** viewer, or an online search engine, or **XQUERY**-style terms evaluated vis-à-vis **JATS** files — index metadata can be consulted just in case a search term overlaps with headings already included in an index. Such functionality may be augmented by extending index metadata to include concepts related to index entries even if those secondary concepts are not indexed themselves. In effect, index metadata may declare that all locators modeled as relevant for their specific entry are also relevant to some list of associated concepts, such that whenever those latter concepts are targeted in a query the explicit locators are included among the query results.

**Series or Archival Meta-Indexes** Once a standard format is developed for indexes in some document-collection, it becomes possible to pool multiple indexes into a common resource. Each time a new publication is finalized, all of its index terms could be inserted into a bibliographic database allowing users to search across multiple books/manuscripts in a given book series, journal collection, or document repository. Such meta-indexes can support searches that are more precise and granular than conventional search-engine technologies based on occurrence vectors and stemming/lemmatization.

**Academic Search Engines** In online contexts, “indexing” typically refers not to human-visible book indexes but rather to compressed data files that catalog nontrivial words and concepts in a manuscript. Such indexing is typically non-contextual; for instance, it is not represented whether a word/phrase is present in text with a special purpose (quotes, footnotes, chapter/section titles, etc.) and discursive roles are ignored (e.g., a document may be identified as *about* some topic, but more specific criteria such as arguing *for*, *against*, or neutrally *evaluating* theories or claims are not modeled). Granularity may be limited by the scope of an archive, because efficient queries can only expend a few seconds per document if searches are to be evaluated against a large collection.

Nonetheless, search functionality can take steps to increase query specificity via meta-indexes and related metadata. For one thing, users may specifically consult search areas that are restricted in scope — say, a single University Press, book series, repository (PubMedCentral, for example), or journal collection. Publishers who curate meta-indexes for mid-size collections can support much more detailed evaluations per document; a book series might have several dozen titles, not tens of thousands. **PacTk** support construction of corpora query languages that express granular searches over multiple documents, with similar logic to filtered/contextualized text searches in a single **PDF** document.



Even where queries are executed via conventional vector-database methods, **PacTk** allows authors or digital editors to exercise control over how word-occurrence lists are compiled. For example, **TSOM** objects can include lemmatized word-lists per sentence; authors may then control for circumstances such as two different meanings reduced to the same linguistic stem, which might not be properly ingested by search engines indexing documents through their default algorithms.

## Data Publishing and Multimedia

Contemporary data transparency and “FAIRSharing” (Findable, Accessible, Interoperable, Reusable) standards introduce new challenges for publishers. In this environment, published documents such as **PDFs** serve as only one part of a package of files, alongside research data sets and, often, numerous forms of multimedia content (potentially including audio, video, image series, **3D** graphics, and statistical/data-visualization tools and diagrams).

As a “publishing accelerator” toolkit, **PacTk** has several components to help publishers construct these supplemental materials, including cross-referencing text documents with supplemental data/media assets, and ensuring **FAIR** usability. Here are several components under development:

**Deserialization and Dataset Tools** Individual scientific disciplines are often associated with domain-specific data publishing formats that are paired with dedicated deserialization tools (i.e., computer code which reads raw data files into live memory, without relying on special-purpose software). Such code libraries are often maintained by individual institutions or organizations, such as academic departments, on behalf of a broad research community. Examples include (among many others) and for genomics; **FITS** and **NETCDF** in astronomy and earth sciences; **PDB** and **MOL** in biochemistry; **IFC** in engineering; and for geophysics and oceanography; **CONLL-U** in computational linguistics; **DICOM** and for bioimaging; and **GIS**-indexed attribute layers and shapefiles for geospatial data sets.

Some domain-specific file formats are derived from general-purpose formats such as **CSV**, **XML**, and **JSON**. Others use idiosyncratic text and/or binary serialization protocols, so that custom parsers are necessary so as to consume byte or character streams from raw data files. Even when the actual encoding is performed via a generic standard like **XML**, dedicated deserialization code will still be needed — generic parsers can convert a given **XML** document to a **DOM** info set, but algorithms will have to navigate the document hierarchy and extract text strings from element nodes, which become the input for initializing object data fields.

**PacTk** seeks to address these concerns by offering a platform-independent Virtual Machine that is easy to embed in scientific applications or build as a standalone program. Scripting or query language can then emit code for this specific **VM** to take responsibility for deserializing encoded data sets. Depend on the format, the **VM** instructions might navigate through graph-like structures in a standardized profile (like **XML**) or implement and/or integrate with (via postprocessors or rule-match callbacks) parsers for custom input file types.

Via these technologies, publishers can ensure that data sets are accessible so long as there are deserialization libraries targeting a **PacTk**, rather than duplicating effort and requiring multiple libraries for different operating systems and programming environments.

**Analytic and Demonstrative Code** Many data sets are published alongside copies or samples of code used to analyze the raw data. Code sharing is important for transparency (allowing outsiders to double-check the accuracy of calculations and conclusion drawn therefrom) as well as reuse.

Although such accompanying code may be implemented in many different languages and environments, the **VM** principles identified above for deserialization tasks also apply to code specific to research data sets. Here, too, accessibility implies that dataset code should be available in a cross-platform manner. Moreover, implementations should proceed via conventions suitable for **GUI** wrappers and interop, because research methods will often depend on domain-specific scientific or academic software. Ideally, data sets will be examined via plugins specific to applications familiar to the relevant research community, or at least custom-built software that can interoperate with domain-specific technologies. Unfortunately, many of the most popular scripting formats in scientific/research contexts (e.g., Python, JavaScript, and **R**) have noticeable limitations vis-à-vis native-**GUI** integration.

**PacTk** enables a different approach where languages can be customized to individual data sets and target pure-**C++ VMs** whose entire parsing, , and runtime stack can be embedded (as source code) in **C++ GUI** applications with no outside dependencies. Compilers and runtime engines for dataset code may thereby be released as part of the publication package itself. The **VM** and peer components could also be built directly in **PDF** viewers, multimedia applications (for videos, digital maps, etc.) and scientific computing platforms.

Another **PacTk** benefit for dataset code involves documentation and pedagogical features. In these contexts, code serves not merely to derive specific computations but also to demonstrate the acquisition methods and analytic protocols relevant for a research project. Such details as units of measurement, expected ranges, typed coordinate systems (e.g., ensuring that procedures expecting horizontal-then-vertical coordinate pairs are never called with the converse), compile-time/statically-analyzable pre- and post-conditions, and so forth, help dataset code to serve as a resource clarifying scientific principles and theoretical models that drive published findings. The **PacTk** machine includes overload-by-contract, dependent-typing (via a framework compatible with template metaprogramming),

native interop (with dynamically loaded as well as statically compiled libraries) and “user-defined” calling convention features that differ in some technical details from typical language interpreters, resulting in a scripting environment specifically built for open-access data sharing and the deployment of data packages as pedagogical tools.

**Full-Text Query Languages** As discussed earlier in this document, **PacTk** can be utilized as a code library by calling methods on **C++** (specifically, **TSOM**) objects. At the same time, a **PacTk** virtual machine supplies an environment for compiling scripting or query languages. These functionalities can be merged, such that the process of extracting information from a **PDF** manuscript, meta-indexed repository, or bibliographic database might be achieved by evaluating formulating part of the code in a specialized query language. The techniques involved would be similar to employing **XQUERY** or **XQSE** expressions to work with **JATS** or **BITS** files.

**PacTk** features its own flexible front-end parsing framework (mentioned above relative to **GTagML**) that could be used to program query grammars in an extensible, adaptable manner. By leveraging full regular expression capabilities, callbacks to arbitrary code (instead of generative production rules), and directly-executable grammar files (with no separate compilation steps), the **PacTk** parsers are more flexible than conventional **/-style** compilers. Meanwhile, the **VM** runtimes allows query evaluators to recognize basic scripting constructions such as locally-scoped variables and calls to arbitrary boolean-valued procedures as filter parameters.

In totality, such features allow text/bibliographic query systems to be implemented in a self-contained and cross-platform manner, with customizations specific to individual publishers and even to each particular journal or book series.

## Science and Humanities GUIs

### Texture Annotations

courts may let this little girl fie k – lisa steinberg horror k – state-pol-act- Ohel-Agency-Under-Baby-Selling-Probe

---

### *Software, Cloud, and GUI Modules*

---

### Data Sets, Queries, and Database Architecture

---

### *Some References*

---



---

### *Video Links*

---

For more information, please see our software demo vidoes: